Accelerating Test-Time Discovery in Complex Code Generation via Execution-Guided Self-Distillation

Mai Phong Đăng

VNUHCM University of Science
Supervisor: Dr. Ngô Minh Mẫn

July 2026

The problem

Test-time training (TTT): fix one hard programming problem, let the model improve itself over a few inference-time steps, then reset.

- Reward is verifiable but sparse: it only says *whether* an attempt passed.
- On-policy self-distillation stalls on hard problems — the bare student rarely produces a correct attempt to learn from.
- This is the **flat-reward trap**: no correct rollout $\Rightarrow$ no usable gradient.

*Goal: accelerate test-time **discovery** on complex code generation.*

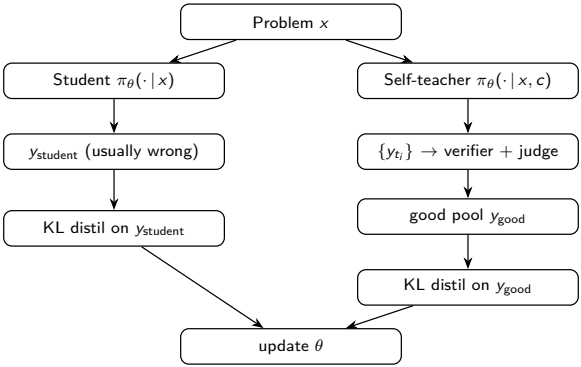
Background: SDPO and the gap

SDPO (Hübotter et al.): extract a dense signal from the *rich feedback* that verifiable environments already produce (failing tests, runtime errors).

- The same model, conditioned on feedback, is a **self-teacher**; its per-token distribution is distilled into the student.
- Denser credit assignment than GRPO: a K-dim target per token, not one scalar per sequence.

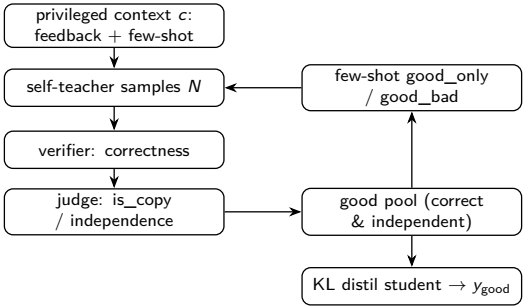
The gap: the parent method is student-first — it distills the student's *own failed rollout*. On hard problems there is little that is correct to learn from.

Our idea: teacher-first



Teacher-first: distil a *correct, filtered* trajectory the teacher generates — between on-policy SDPO and off-policy SFT.

The teacher-first step



Verifier + judge filter for *correct AND independent*; the filter never distils a copy — when every candidate is a copy, the good pool is empty and that step distils nothing.

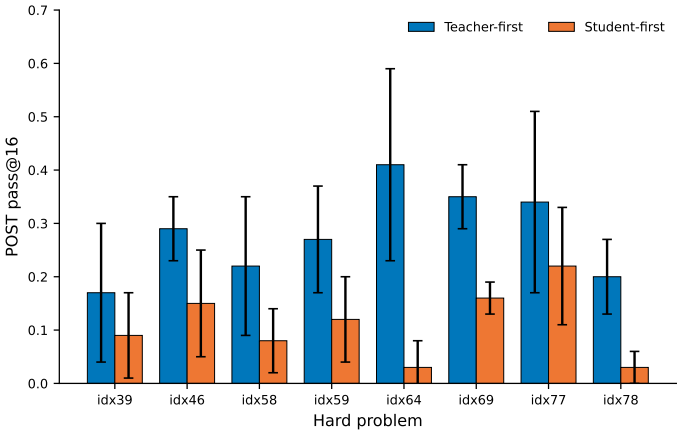
Research objectives

- **RO-method:** Establish whether teacher-first beats student-first at discovering hard code solutions — and whether it holds at *matched compute*.
- **RO1:** Determine whether the *reprompt-template* formulation of the feedback affects discovery.
- **RO2:** Measure whether distillation from rich context *suppresses epistemic verbalization* on code.
- **RO3:** Characterize the *structural & domain limitations* going from procedural code to value-only math.

Setup

- Model: **Qwen3-4B** (LoRA), thinking ON — the *same* model on both code (LiveCodeBench v6) and math (AIME 2026 pilot).
- Per problem: reset to base, **15 TTT steps**, evaluate student PRE vs POST.
- **8 frontier code problems** $\times$ **6 seeds** (medium scale) $\Rightarrow$ Wilcoxon signed-rank test.
- Metrics: pass@16, discovery@k, total generations + GPU-time.

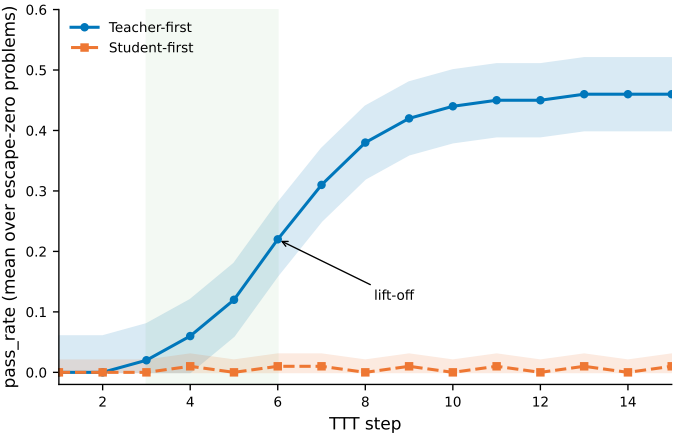
RO-method: teacher-first wins



Mean over 6 seeds; error bars = 1 SD. Wilcoxon signed-rank $p < 0.01$.

- TF $>$ SF on every problem (mean), largest margins on the hardest — **39 / 8 / 1** over 48 seed-pairs.
- Wilcoxon $p < 0.01$ (8 problems $\times$ 6 seeds) — not a crude sign test.
- **RO1:** on hard problems, template order **T5** $>$ **T2** $>$ **T1** (reasoning-inducing best).

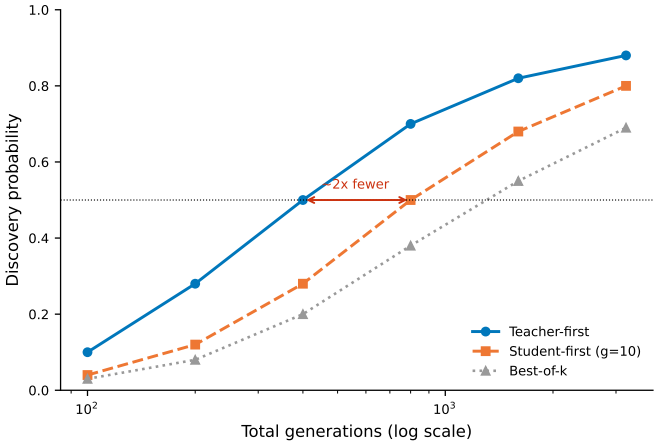
The headline: escape-zero



Student-first stays flat at 0; teacher-first escapes from step ∼4.

- Student-first stays flat at zero.
- Teacher-first **lifts off** from ∼step 4.
- Direct signature of the flat-reward trap: on-policy has nothing correct; off-policy injection does.

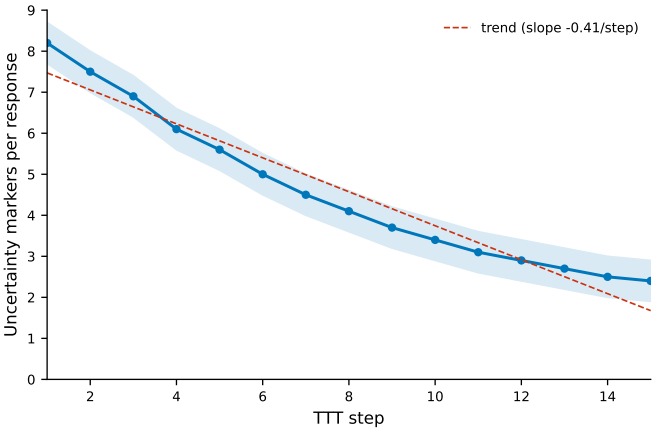
Compute-fair: not just more sampling (RO-method)



Compute-to-correct frontier: TF reaches a target discovery at ~2x fewer generations.

- Match the budget: student-first at **10 generations/step**.
- Teacher-first still wins, and reaches a target discovery with $\sim 2\times$ fewer total generations.
- The advantage is the *kind* of distilled trajectory, not sample volume.

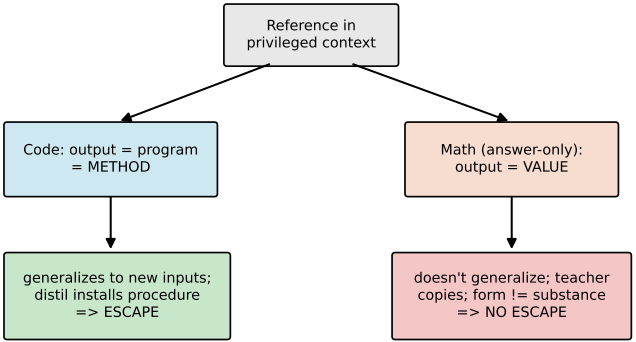
RO2: epistemic verbalization on code



Epistemic markers decline and accumulate downward across TTT steps (code, thinking ON).

- Uncertainty markers ("wait", "let me reconsider") **decline** across steps, accumulating.
- Consistent with Kim et al.: high $I(y; c \mid x) \Rightarrow$ suppression.
- *On code* it co-occurs with improved correctness — read as consolidation (a *hypothesis*; ref-type ablation is future work).

RO3: value vs. procedure — when it works



- Code output *is* a method (a program that generalizes) ⇒ distillation installs a procedure ⇒ **escape**.
- Answer-only math is a value ⇒ teacher can only copy ⇒ **no escape**.
- *Same* Qwen3-4B on both ⇒ not a weaker model; reference type is the *primary* difference (budget also differs) — n=2 pilot, read as directional.

Contributions

1. A **teacher-first** organization of execution-guided self-distillation (clean filter, never distils a copy).
2. Teacher-first **significantly accelerates discovery** and **escapes the flat-reward trap** — compute-fair (RO-method).
3. A **reprompt-template effect**, strongest on hard problems (RO1).
4. A measured **epistemic-suppression** result on code (RO2).
5. A **value-versus-procedure boundary** characterizing *when* the method helps.

Limitations & future directions

Honest bounds (§6.2):

- One **4B** Qwen model at a personal compute scale — a characterization, not a scaling law.
- Math pilot bottlenecked by **AIME**'s answer-only feedback (no worked solutions to derive from); n=2.
- A stronger base (**8B+**) is expected to *narrow* the TF-vs-SF margin on reachable tasks.
- Epistemic result tied to one model / lexical marker set.

Future directions: method-bearing math references (**MATH-500**); a same-domain *reference-type ablation* to test the boundary causally; scale to **8B+** and a second code benchmark.

Conclusion

- **Teacher-first execution-guided self-distillation** accelerates test-time discovery in complex code generation.
- It escapes the flat-reward trap, the advantage is **compute-fair**, and it shows a regime-dependent epistemic effect.
- The **value-versus-procedure boundary** explains *when* it works: an in-reach method, not just an answer.

Thank you

for your attention.